

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/272946925>

Fast GPU integration algorithm for isogeometric finite element method solvers using task dependency graphs

Article in *Journal of Computational Science* · February 2015

DOI: 10.1016/j.jocs.2015.02.007

CITATIONS

16

READS

2,082

1 author:



Maciej Woźniak

AGH University of Krakow

39 PUBLICATIONS 318 CITATIONS

SEE PROFILE

Fast GPU integration algorithm for isogeometric finite element method solvers using task dependency graphs

Maciej Woźniak^a

^a*AGH University of Science and Technology, Krakow, Poland*

Abstract

This paper analyzes the integration for isogeometric finite element method solvers. In particular, it shows that isogeometric solvers with higher order B-splines spend significant amount of time for generation of the element frontal matrices when executed sequentially on CPU. The integration algorithm is represented as a sequence of basic undividable computational tasks and the dependency relation between them is identified. The basic tasks are defined for particular steps of the integration algorithm, for given integration points. In this paper we show how to prepare independent sets of tasks that can be automatically scheduled and executed concurrently in a GPU card. This is done with the help of the graph expressing the dependency between tasks, constructed for the integration algorithm. The algorithm is implemented on GPU and tested on a sequence of numerical examples concerning the two dimensional isogeometric L^2 -projection problems for the MRI scans of the human head. The execution time of the concurrent GPU integration is compared with the sequential integration executed on CPU.

Keywords: isogeometric finite element method, numerical integration, L^2 -projection, modeling of brain

1. Introduction

Finite element computations with higher order basis functions involve two phases: (1) generation of the system of linear equations, distributed into multiple element frontal matrices and (2) execution of an external solver algorithm of the global system of linear equations. There are several higher order basis functions that can be utilized for an approximation of finite element solution, e.g. the hierarchical basis functions used by *hp*-adaptive

finite element methods [3, 4], or B-splines used in isogeometric finite element method [1, 2]. Classical higher order finite element methods with hierarchical basis functions deliver C^0 continuity on the interfaces between elements [3, 4] and C^p continuity inside of elements [3, 4]. (Actually, we have C^∞ continuity inside the elements, since 0 is infinitely continuous function). The isogeometric finite element method with B-spline basis functions delivers global C^{p-1} continuity [5, 6]. (Again, we actually have C^∞ global continuity inside the elements, since higher order derivatives are equal to 0, which is an infinitely continuous function).

The generated system of linear equations for elliptic problems is usually solved with either multi-frontal direct solvers [11, 12] such as MUMPS [13, 14, 15, 16], SuperLU [20, 21] or PaStiX [22], or iterative solvers, such as the ones available through PETSc library [23, 24, 25]. There are also some modern linear computational cost solvers utilizing the concept of H-matrices, delivering approximate solutions [29, 30]. It is also possible to obtain linear computational cost for two dimensional h -refined grids with point or edge singularities [27, 26, 28].

The integration for higher order basis functions executed sequentially on a CPU is often expensive. In general, for higher order basis functions, there are $O(p)$ functions per single element in one dimensional problems ($1D$), and the integration requires $O(p^3)$ operations. In two dimensions ($2D$), integration requires $O(p^6)$ operations and in three dimensions ($3D$), it requires $O(p^9)$ operations over a single element.

There are some sum factorization techniques for speeding up the integrations, like the one proposed in [18] for $1D$ problems. Another method applies for $2D$ and $3D$ hierarchical basis functions and they may reduce the computational cost from $O(p^6)$ down to $O(p^5)$ in $2D$ and from $O(p^9)$ down to $O(p^7)$ (see [3, 4]).

However, the introduction of sum factorization techniques makes our analysis more difficult, since the dependency between tasks is complicated. Additionally, for efficient GPU computations, it is necessary to ensure that sets of tasks scheduler over GPU nodes to be executed concurrently are identical tasks working for different input data. Moreover, the sum factorization techniques are dependent on the tensor product structure of the basis functions used and we are not aware of sum factorization algorithms for B-splines. Thus, we restrict our research to the case without the sum factorization and we may postpone the sum factorization techniques to future work.

While for the multi-frontal direct solver it has been already shown that

concurrent GPU execution with graph grammar based approach can reduce significantly the complexity pattern [7], a fast concurrent integration algorithm is proposed. This is done by localizing basic undividable tasks and finding sets of tasks that can be executed concurrently [8].

Fast quadrature algorithms for GPU or multi-core CPU have been already proposed based on OMP loop parallelization and careful analysis of the processor architecture for the hierarchical basis functions [9, 10]. In this paper, we focus on the selection of tasks for the integration algorithm and construction of the graph tasks showing the dependency between them. In particular, we show how to prepare sets of independent tasks representing particular steps of the integration process. These tasks are then automatically scheduled by the GPU controller and executed concurrently, set by set.

We concentrate on B-spline basis functions employed in a $2D$ isogeometric finite element method. We restrict our considerations to $2D$ isogeometric L^2 -projection problem. This is just an exemplary simple model problem and the analysis presented can be applied to any elliptic problem in $2D$ with analogous results. The algorithm is tested on the isogeometric $-L^2$ -projection performed for the MRI scans data of the human head. The execution time of integration performed on CPU is compared with the integration performed on GPU.

There are many different quadrature rules delivering different accuracies of the numerical integration. In particular, for B-spline basis functions, there have been some new faster quadrature rules recently proposed in [19]. However, in this paper we focus on classical Gaussian quadrature, but our results can also be applied to other more advanced quadrature rules, since the selection and construction of tasks graph expressing the dependency between tasks does not depend on the selected quadrature rule.

2. The integration algorithm

2.1. Overview

This section presents a weak form of the classical L^2 -projection problem, used as a model problem to test the efficiency of our concurrent integration algorithm. The L^2 -projection problem in the weak form is the following: We are looking for the $u \in V \subset L^2$, being the orthogonal L^2 -projection of piecewise constant function F from $L^2(\Omega)$ (representing pointwise MRI scan data) on the space of B-splines V , which results in the condition $(F - u, v -$

$u) = 0, \forall v \in V$, where $(L^2(\Omega))^2 \ni (u, v) \rightarrow \int_{\Omega} u v dx \in \mathbb{R}$ stands for the scalar product in $L^2(\Omega)$. Observing that $V - u = V$ we can obtain the L^2 -projection problem in the more convenient form:

$$(u, v) = (F, v) \forall v \in V, \quad V = \text{span}\{B_{i,j;p}\}_{i=1,\dots,N_x,j=1,\dots,N_y} \quad (1)$$

where $B_{i,j;p}$ are B-spline basis functions of a given order p [1]. The B-spline order p is uniform, and constant for all basis functions.

If we select a basis $\{B_{i,j;p}\}_{i=1,\dots,N_x,j=1,\dots,N_y}$ in V , then an arbitrary $v \in V$ can be represented by $v = \sum_{k=1,\dots,N_x,l=1,\dots,N_y} B_{k,l;p} b_{k,l}$ and the L^2 -projection problem can be rewritten in the form

$$\sum_{k=1,\dots,N_x,l=1,\dots,N_y} b_{k,l} (u - F, B_{k,l;p}) = 0 \quad \forall \{b_{k,l}\} \quad (2)$$

Because $\{b_{k,l}\}$ is arbitrary, then (2) is satisfied if and only if $R^{N_x N_y} \ni (u - F, B_{k,l;p}) = 0$ as the unique vector in $R^{N_x N_y}$ orthogonal to all others. Then, we obtain a system of equations

$$(u, B_{k,l;p}) = (F, B_{k,l;p}) \quad k = 1, \dots, N_x, l = 1, \dots, N_y \quad (3)$$

and finally using the representation of

$$u = \sum_{i,j} B_{i,j;p} a_{i,j} \quad (4)$$

we obtain

$$\sum_{i=1,\dots,N_x,j=1,\dots,N_y} a_{i,j} (B_{i,j;p}, B_{k,l;p}) = (F, B_{k,l;p}) \quad k = 1, \dots, N_x, l = 1, \dots, N_y \quad (5)$$

Using Gaussian quadrature, and the definition of $B_{k,l;p}(x_1, x_2) = N_{k;p}(x_1)N_{l;p}(x_2)$, the definition of the scalar product the integration over the domain can be presented by a weighted summation over Gaussian points.

$$\begin{aligned} & \int_{\Omega} N_{i;p}(x_1) N_{j;p}(x_2) N_{k;p}(x_1) N_{l;p}(x_2) dx_1 dx_2 \\ &= \sum_n w_n N_{i;p}(x_1^n) N_{j;p}(x_2^n) N_{k;p}(x_1^n) N_{l;p}(x_2^n) \\ & \quad \forall i, k = 1, \dots, N_x, j, l = 1, \dots, N_y \end{aligned} \quad (6)$$

and

$$\int_{\Omega} N_{k;p}(x_1) N_{l;p}(x_2) dx_1 dx_2 = \sum_n w_n N_{k;p}(x_1^n) N_{l;p}(x_2^n) \quad \forall k = 1, \dots, N_x, l = 1, \dots, N_y \quad (7)$$

for a given p , where (x_n^1, x_n^2) and w_n denotes the Gaussian quadrature integration points and weights. For interfacing with either a direct or an iterative solver, the mesh is partitioned into elements, as illustrated in Figure 1. Next, element frontal matrices are generated by performing integration over particular elements.

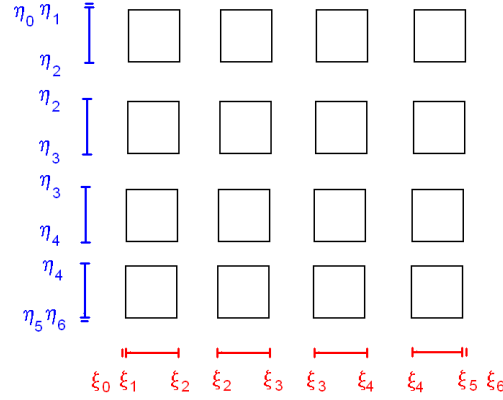


Figure 1: Partition of the computational mesh into elements

2.2. Linear basis functions

For linear basis functions we take $2 * 2 = 4$ two dimensional B-splines, each of which is tensor product of two one dimensional B-splines. This is illustrated in Figure 2.

A frontal matrix for the case of linear basis functions is illustrated in Table 1. The frontal matrix is obtained from integration over a single finite element $E_{k,l}$, where four basis functions have non-zero support, namely $B_{k-1,l-1;1}$, $B_{k,l-1;1}$, $B_{k-1,l;1}$, $B_{k,l;1}$. Thus, the rows and columns in the frontal matrix correspond to the four basis functions, and its entries contain scalar products of all the combinations of the basis functions. In other words, indices $k, k-1, l, l-1$ are used for element $E_{k,l}$ relative numbering of basis functions. The considerations presented here are identical for all elements

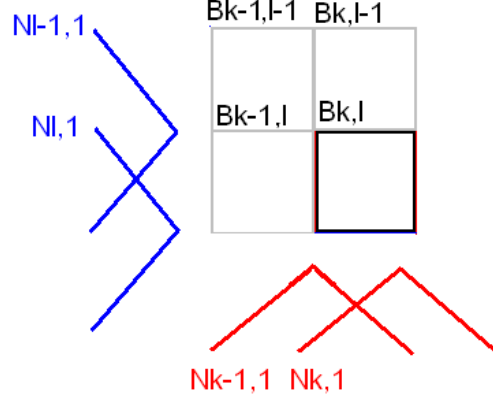


Figure 2: Partition of the computational mesh into elements

$(B_{k-1,l-1;1}, B_{k-1,l-1;1})$	$(B_{k,l-1;1}, B_{k-1,l-1;1})$	$(B_{k-1,l;1}, B_{k-1,l-1;1})$	$(B_{k,l;1}, B_{k-1,l-1;1})$
$(B_{k-1,l-1;1}, B_{k,l-1;1})$	$(B_{k,l-1;1}, B_{k,l-1;1})$	$(B_{k-1,l;1}, B_{k,l-1;1})$	$(B_{k,l;1}, B_{k,l-1;1})$
$(B_{k-1,l-1;1}, B_{k-1,l;1})$	$(B_{k,l-1;1}, B_{k-1,l;1})$	$(B_{k-1,l;1}, B_{k-1,l;1})$	$(B_{k,l;1}, B_{k-1,l;1})$
$(B_{k-1,l-1;1}, B_{k,l;1})$	$(B_{k,l-1;1}, B_{k,l;1})$	$(B_{k-1,l;1}, B_{k,l;1})$	$(B_{k,l;1}, B_{k,l;1})$

Table 1: Frontal matrix with linear basis functions and the corresponding tasks names

$E_{k,l}$. Each entry of the frontal matrix is obtained by summing up the values of the scalar products at Gaussian integration points. The computational tasks of evaluating the value of the scalar product at Gaussian integration point is presented in Table 2 and denoted by $t_{i,j;k,l}^1$.

In such case, it is necessary to compute $2 * 2 = 4$ linear basis functions at Gaussian integration points, as presented in Table 3. Again, in this table we have named particular tasks by $t_{i,j}^1$.

The computational tasks of evaluating the linear basis functions involve tensor products of $2 + 2 = 4$ one dimensional linear B-splines. This is represented in Table 4, where we have named particular tasks by t_i^1 .

2.3. Quadratic basis functions

For quadratic basis functions, we have $3 * 3 = 9$ tensor products of two one dimensional B-splines. This is illustrated in Figure 3.

A frontal matrix for the case of quadratic basis functions is illustrated in Table 5. The frontal matrix is obtained from integration over a single fi-

$t_{k-1,l-1;k-1,l-1}^1 =$ $(B_{k-1,l-1;1}(x_1, x_2), B_{k-1,l-1;1}(x_1, x_2))$	$t_{k,l-1;k-1,l-1}^1 =$ $(B_{k,l-1;1}(x_1, x_2), B_{k-1,l-1;1}(x_1, x_2))$
$t_{k-1,l;k-1,l-1}^1 =$ $(B_{k-1,l;1}(x_1, x_2), B_{k-1,l-1;1}(x_1, x_2))$	$t_{k,l;k-1,l-1}^1 =$ $(B_{k,l;1}(x_1, x_2), B_{k-1,l-1;1}(x_1, x_2))$
$t_{k-1,l-1;k,l-1}^1 =$ $(B_{k-1,l-1;1}(x_1, x_2), B_{k,l-1;1}(x_1, x_2))$	$t_{k,l-1;k,l-1}^1 =$ $(B_{k,l-1;1}(x_1, x_2), B_{k,l-1;1}(x_1, x_2))$
$t_{k-1,l;k,l-1}^1 =$ $(B_{k-1,l;1}(x_1, x_2), B_{k,l-1;1}(x_1, x_2))$	$t_{k,l;k,l-1}^1 =$ $(B_{k,l;1}(x_1, x_2), B_{k,l-1;1}(x_1, x_2))$
$t_{k-1,l-1;k-1,l}^1 =$ $(B_{k-1,l-1;1}(x_1, x_2), B_{k-1,l;1}(x_1, x_2))$	$t_{k,l-1;k-1,l}^1 =$ $(B_{k,l-1;1}(x_1, x_2), B_{k-1,l;1}(x_1, x_2))$
$t_{k-1,l;k-1,l}^1 =$ $(B_{k-1,l;1}, B_{k-1,l;1}(x_1, x_2))$	$t_{k,l;k-1,l}^1 =$ $(B_{k,l;1}, B_{k-1,l;1}(x_1, x_2))$
$t_{k-1,l-1;k,l}^1 =$ $(B_{k-1,l-1;1}(x_1, x_2), B_{k,l;1}(x_1, x_2))$	$t_{k,l-1;k,l}^1 =$ $(B_{k,l-1;1}(x_1, x_2), B_{k,l;1}(x_1, x_2))$
$t_{k-1,l;k,l}^1 =$ $(B_{k-1,l;1}(x_1, x_2), B_{k,l;1}(x_1, x_2))$	$t_{k,l;k,l}^1 =$ $(B_{k,l;1}(x_1, x_2), B_{k,l;1}(x_1, x_2))$

Table 2: Computational tasks responsible for evaluation of the values of scalar products of two dimensional linear basis functions over element $E_{k,l}$ at Gaussian quadrature points.

nite element $E_{k,l}$, where nine basis functions have non-zero support, namely $B_{k-2,l-2;2}, \dots, B_{k,l;1}$. Thus, the rows and columns in the frontal matrix corresponds to the nine basis functions, and its entries contain scalar products of all the combinations of the basis functions. In other words, indices $k, k-1, k-2$ and $l, l-1, l-2$ are used for element $E_{k,l}$ relative numbering of basis functions. The considerations presented here are identical for all elements $E_{k,l}$. The polynomial order of approximation $p = 2$ is fixed. Each entry of the frontal matrix is obtained by summing up the values of the scalar products at Gaussian integration points. The computational tasks of computing the value of the scalar product at Gaussian integration point is presented in Table 6 and denoted by $t_{i,j;k,l}^2$.

In such case, it is necessary to compute $3 \times 3 = 9$ quadratic basis functions at Gaussian quadrature points, presented in Table 7, with tasks $t_{i,j}^2$.

The computational tasks of computing the quadratic basis functions involve tensor products of $3 + 3 = 6$ one dimensional quadratic B-splines at the points, as presented in Table 8. The tasks are named as t_i^2 .

$t_{k,l}^1 = B_{k,l;1}(x_1, x_2) = N_{k;1}(x_1)N_{l;1}(x_2)$
$t_{k,l-1}^1 = B_{k,l-1;1}(x_1, x_2) = N_{k;1}(x_1)N_{l-1;1}(x_2)$
$t_{k-1,l}^1 = B_{k-1,l;1}(x_1, x_2) = N_{k-1;1}(x_1)N_{l;1}(x_2)$
$t_{k-1,l-1}^1 = B_{k-1,l-1;1}(x_1, x_2) = N_{k-1;1}(x_1)N_{l-1;1}(x_2)$

Table 3: Computational tasks responsible for evaluation of the values of two dimensional linear basis functions over element $E_{k,l}$ at Gaussian quadrature points.

$t_k^1 =$ $N_{k;1}(x_1)$	$t_{k-1}^1 =$ $N_{k-1;1}(x_1)$
$t_l^1 =$ $N_{l;1}(x_2)$	$t_{l-1}^1 =$ $N_{l-1;1}(x_2)$

Table 4: Computational tasks responsible for evaluation of the values of one dimensional linear basis functions over element $E_{k,l}$ at Gaussian quadrature points.

2.4. Higher order basis functions

The scheme presented in subsection 2.2 and 2.3 for linear and quadratic basis functions, respectively, can be generalized for arbitrary higher order basis functions.

In particular, over a single element $E_{k,l} = [\xi_K, \xi_{K+1}] \times [\eta_L, \eta_{L+1}]$ there are $(p+1)(p+1)$ basis functions defined as tensor products of one dimensional B-splines of order p

$$\{B_{m,n;p}(x_1, x_2)\}_{m=k-p,\dots,k;n=l-p,\dots,l} = \{N_{m;p}(x_1)N_{n;p}(x_2)\}_{m=k-p,\dots,k;n=l-p,\dots,l} \quad (8)$$

so, it is necessary to compute their values at Gaussian quadrature integration points.

3. Selection of tasks and construction of tasks graph for the integration algorithm

3.1. Linear basis functions

Let us focus first on the linear basis functions case. We identify basic computational tasks for the integration algorithm and construct the alphabet of defined tasks as the computations performed by these tasks on actual data,

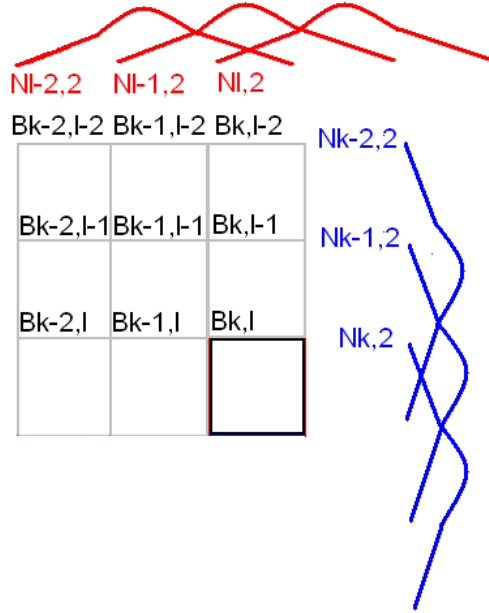


Figure 3: Quadratic B-splines over a single element

$(B_{k-2,l-2;2}, B_{k-2,l-2;2})$	$\dots$	$(B_{k,l;2}, B_{k-2,l-2;2})$
$\dots$	$\dots$	$\dots$
$(B_{k-2,l-2;2}, B_{k,l;2})$	$\dots$	$(B_{k,l;2}, B_{k,l;2})$

Table 5: Frontal matrix with quadratic basis functions

in our case, on the integration points. The analysis presented in this section follows the theoretical plot presented by the trace theory [8].

Definition 1. *The alphabet of tasks for linear B-spline based basis functions consists of the particular computational tasks executed during the integration process for linear B-spline basis functions, for a given data, namely for a given integration point:*

- Computations of the entries of the frontal matrix, e.g. $t_{k,l;k,l}^1 = (B_{k,l;1}, B_{k,l;1})$ as expressed in Table 2,
- Computations of the values of linear basis functions at Gaussian integration points, e.g. $t_{k,l}^1 = B_{k,l;1}(x_1, x_2)$ as expressed in Table 3,

$t_{k-2,l-2;k-2,l-2}^2 =$ $(B_{k-2,l-2;2}(x_1, x_2), B_{k-2,l-2;2}(x_1, x_2))$	$\dots$	$t_{k,l;k-2,l-2}^2 =$ $(B_{k,l;2}(x_1, x_2), B_{k-2,l-2;2}(x_1, x_2))$
$\dots$	$\dots$	$\dots$
$t_{k-2,l-2;k,l}^2 =$ $(B_{k-2,l-2;2}(x_1, x_2), B_{k,l;2}(x_1, x_2))$	$\dots$	$t_{k,l;k,l}^2 =$ $(B_{k,l;2}(x_1, x_2), B_{k,l;2}(x_1, x_2))$

Table 6: Computational tasks responsible for evaluation of the values of scalar products of two dimensional quadratic basis functions over element $E_{k,l}$ at Gaussian quadrature points.

$t_{k,l}^2 = B_{k,l;2}(x_1, x_2)$ $= N_{k;2}(x_1)N_{l;2}(x_2)$	$t_{k,l-1}^2 = B_{k,l-1;2}(x_1, x_2)$ $= N_{k;2}(x_1)N_{l-1;2}(x_2)$	$t_{k,l-2}^2 = B_{k,l-2;2}(x_1, x_2)$ $= N_{k;2}(x_1)N_{l-2;2}(x_2)$
$t_{k-1,l}^2 = B_{k-1,l;2}(x_1, x_2)$ $= N_{k-1;2}(x_1)N_{l;2}(x_2)$	$t_{k-1,l-1}^2 = B_{k-1,l-1;2}(x_1, x_2)$ $= N_{k-1;2}(x_1)N_{l-1;2}(x_2)$	$t_{k-2,l-1}^2 = B_{k-1,l-2;2}(x_1, x_2)$ $= N_{k-1;2}(x_1)N_{l-2;2}(x_2)$
$t_{k-2,l}^2 = B_{k-2,l;2}(x_1, x_2)$ $= N_{k-2;2}(x_1)N_{l;2}(x_2)$	$t_{k-2,l-1}^2 = B_{k-2,l-1;2}(x_1, x_2)$ $= N_{k-2;2}(x_1)N_{l-1;2}(x_2)$	$t_{k-2,l-2}^2 = B_{k-2,l-2;2}(x_1, x_2)$ $= N_{k-2;2}(x_1)N_{l-2;2}(x_2)$

Table 7: Computational tasks responsible for evaluation of the values of two dimensional quadratic basis functions over element $E_{k,l}$ at Gaussian quadrature points.

- *Computations of the values of one dimensional B-spline basis functions values at Gaussian integration points, e.g. $t_k^1 = N_{k;1}(x_1)$ as expressed in Table 4.*

The generation of the frontal matrix involves the computation of the values of scalar products $(B_{k,l;1}, B_{m,n;1})$ for $k, m = 1, \dots, N_x; l, n = 1, \dots, N_y$. It involves evaluation of the values of the multiplication of the two basis functions at Gaussian integration points. This operation is denoted by $t_{k,l;k,l}^1 = B_{k,l;1}(x_1, x_2)B_{m,n;1}(x_1, x_2)$. This operation, in turn, can be expressed as multiplication of two operations, namely computing the value of $B_{k,l;1}(x_1, x_2; 1)$ and $B_{m,n;1}(x_1, x_2)$. We have denoted these basic operations as $t_{k,l}^1$ and $t_{m,n}^1$. Finally, evaluation of the value of two dimensional B-spline at Gaussian quadrature point can be expressed as multiplication of $N_{k;1}(x_1)$ and $N_{l;1}(x_2)$. We have denoted these tasks by t_k^1, t_l^1 . We can plot the graph presenting the dependency between these tasks. The graph presented in Figure 4 can be understood as a Dickert graph in a sense of the trace theory, compare [8]. The graph can be obtained by considering the representation of

$t_k^2 = N_{k;2}(x_1)$	$t_{k-1}^2 = N_{k-1;2}(x_1)$	$t_{k-2}^2 = N_{k-2;2}(x_1)$
$t_l^2 = N_{l;2}(x_2)$	$t_{l-1}^2 = N_{l-1;2}(x_2)$	$t_{l-2}^2 = N_{l-2;2}(x_2)$

Table 8: Computational tasks responsible for evaluation of the values of one dimensional quadratic basis functions over element $E_{k,l}$ at Gaussian quadrature points.

a trace, in a sense of action of computation of the value at prescribed Gaussian quadrature point. By symbol “...” we denote the fact that we present only a small part of the graph. This is because the graph is huge, since it involves all the Gaussian quadrature points, and all the basis functions.

Finally, by executing the coloring of the Dickert graph (compare [8]), we obtain the sets of tasks that can be executed in parallel. In particular, all the 16 tasks can be executed concurrently.

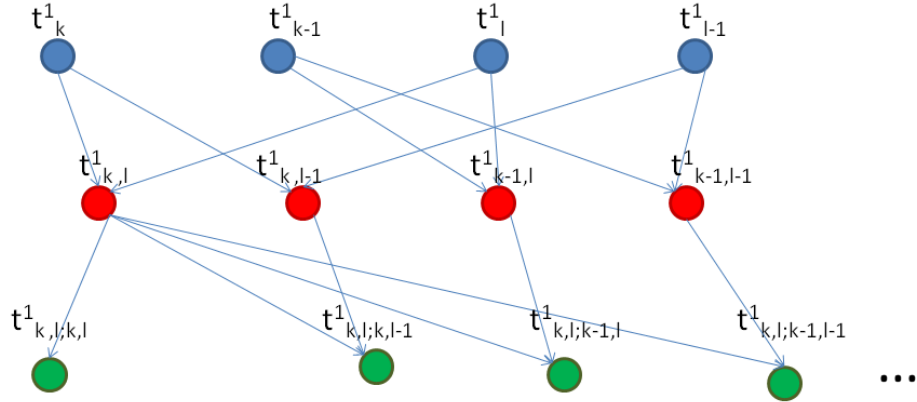


Figure 4: Dickert graph between tasks expressing the integration with linear basis functions

3.2. Higher order basis functions

For higher order basis functions, the analysis is similar to the one performed for the linear case. However, we now need the help of the recursive Cox-de-Boor formula, presented in Figure 5, expressing the higher order B-splines as a product of lower order B-splines. By using this formula, we can express the higher order B-splines as multiplications and additions of lower order B-splines and extend our analysis to higher order cases.

Cox-de Boor formula

$$N_{i,0}(\xi) = I_{[\xi_i, \xi_{i+1}]}$$

$$N_{i,p}(\xi) = \frac{\xi - \xi_i}{\xi_{i+p} - \xi_i} N_{i,p-1}(\xi) + \frac{\xi_{i+p+1} - \xi}{\xi_{i+p+1} - \xi_{i+1}} N_{i+1,p-1}(\xi)$$

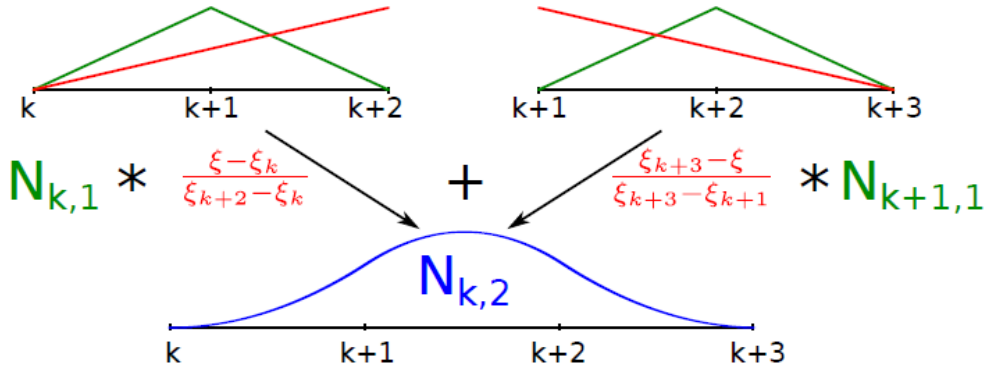


Figure 5: Cox-de-Boor formula

Let us focus on the quadratic basis functions case and perform the task identification again, utilizing the Cox-de-Boor formulae. In other words, we identify the alphabet of tasks, understood as computational tasks performed at given integration points.

Definition 2. *The alphabet of tasks for quadratic B-spline based basis functions consists of:*

- Computations of the entries of the frontal matrix, e.g. $t_{k,l;k,l}^2 = (B_{k,l;2}, B_{k,l;2})$ as expressed in Table 6,
- Computations of the values of quadratic basis functions at Gaussian integration points, e.g. $t_{k,l}^2 = B_{k,l;2}(x_1, x_2)$ as expressed in Table 7,
- Computations of the values of one dimensional second order B-spline basis functions at Gaussian integration points, e.g. $t_k^2 = N_{k;2}(x_1)$ as expressed in Table 8.
- Computations of the values of one dimensional first order B-spline basis functions at Gaussian integration points, e.g. $t_k^1 = N_{k;1}(x_1)$ as

expressed in Table 4.

Again, we introduce the dependency relation between tasks and display a Dickert graph (compare [8]), see Figure 6. At the last level there are $9 \times 9 = 81$ tasks and we only show 9 of them for simplicity of the presentation. The Dickert graph can be colored to obtain the sets of tasks that can be executed in parallel, including all the 81 tasks from the last level in one final set.

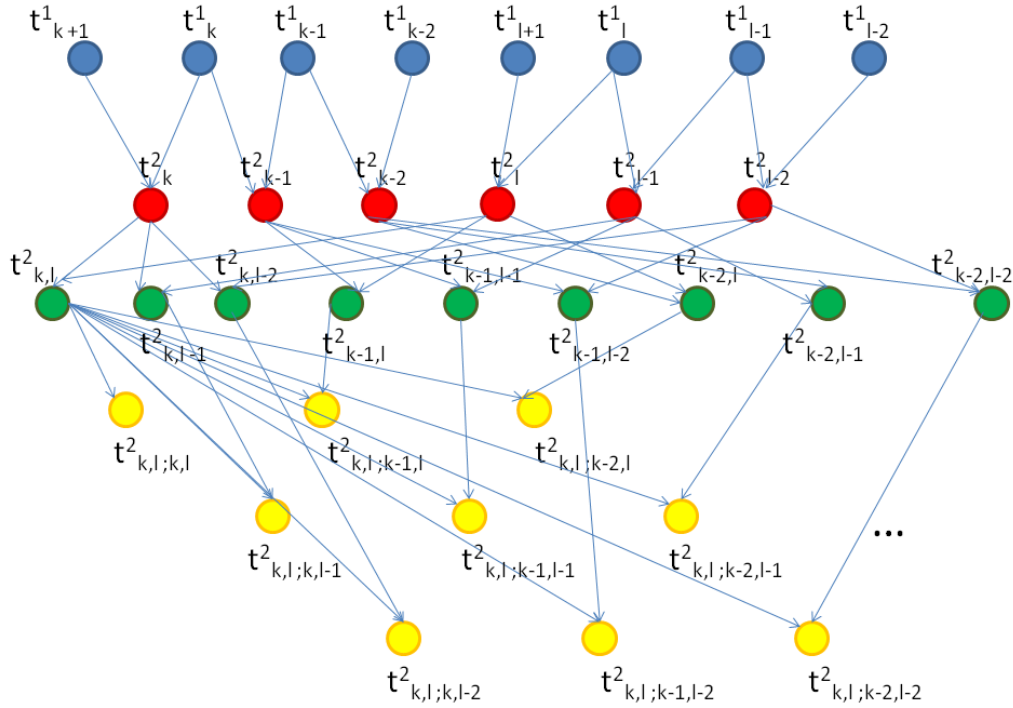


Figure 6: Dickert graph between tasks expressing the integration with quadratic basis functions

4. Numerical results

4.1. Comparison of GPU and CPU integration time

This section presents experimental results for linear, quadratic and cubic B-splines, for 2D isogeometric L^2 -projection problem. The measurements

concern the execution time for sequential integration algorithm executed over CPU and concurrent integration algorithm executed over a shared memory GPU.

The numerical results have been obtained on GeForce GTX 780 graphic card equipped with 3 gigabytes of memory and 2304 cores, as well as on NVIDIA Tesla K20c device, which has 5 gigabytes of memory and 2496 cores.

Computations for CPU integration were performed on a SkyFall linux workstation with Intel Core i7-4820K processor using latest version of PETIGA compiled with intel compiler with the highest level of optimization. The CPU computations have been performed using state-of-the-art PETIGA toolkit with efficient implementation of the integration algorithm. Computations for GTX GPU integration were performed on a SkyFall linux workstation with ASUS GTX-780-DC2OC-3GD5 GeForce GTX 780 graphics card compiled with standard nvcc with standard gcc beneath and the highest level of optimization. Computations for Tesla GPU integration were performed on a Merlin linux server with NVidia Tesla K20c graphics card compiled with standard nvcc with standard gcc beneath and highest level of optimization.

The comparison of the results for linear, quadratic and cubic B-splines are presented in Figures 7, 8 and 9, respectively.

The integration for the isogeometric finite element method in a shared memory graphic card is between one to two orders of magnitude faster than the integration over a single CPU. When we have many cores available and the problem is small enough, we obtain $O(\log N)$ integration time, compare "GPU integration" in Figures 7, 8 and 9. However, when the problem size grows, we obtain the same linear $O(N)$ trend as for the sequential integration. However we can efficiently utilize all the cores, compare GTX and Tesla plots in Figures 7, 8 and 9, for two different GPU's, respectively.

4.2. Exemplary L^2 -projections of the human head

In this section some exemplary isogeometric L^2 -projections of the human head obtained by using our L^2 -projection algorithm with fast integration of GPU are presented. The MRI scans data are presented in Figure 11 and the corresponding L^2 -projections are depicted in Figure 12. The L^2 -projections of the human head presented in this paper can be used to create digital model of the human brain. The numerical technique presented in this paper increases the performance significantly to the point where it is possible to perform modeling of the dynamics of the human head in real time. In particular

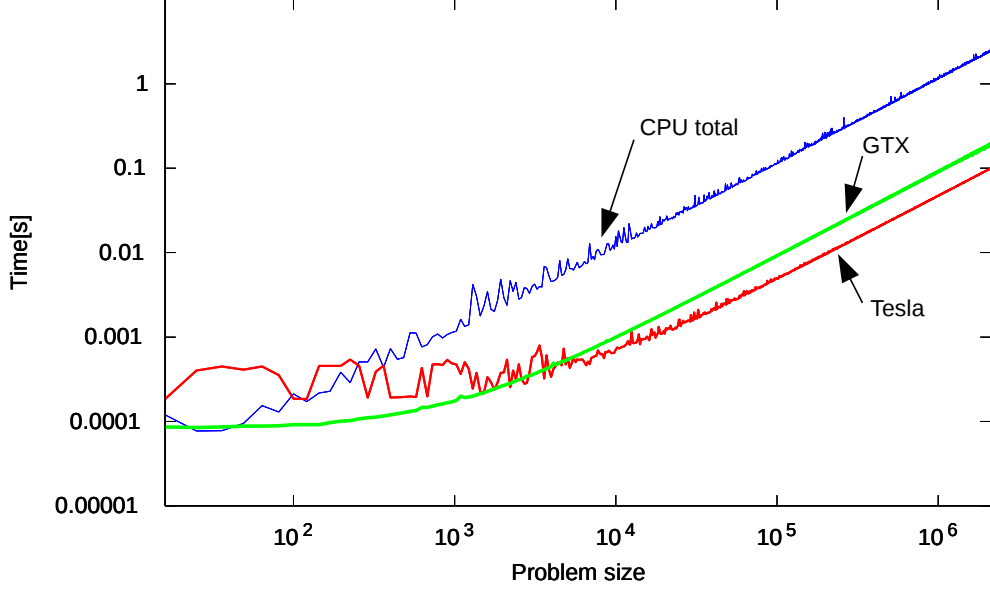


Figure 7: Comparison of GPU and CPU integration time for linear B-splines

the L^2 -projection algorithm can be applied for solution of non-stationary problems, and it may enable on-line medical diagnostic.

5. Conclusions

This paper analyzed the integration algorithm with higher order B-spline basis functions. We provided sets of tasks that can be automatically scheduled and executed concurrently, set by set, on GPU. The concurrent integration algorithm executed on CPU was compared with the integration algorithm execution on GPU. The algorithms were tested on the exemplary two dimensional L^2 -projection problems, however, the construction of the concurrent algorithm remains the same if we replace the L^2 -projection problem with any other two dimensional elliptic problem. The concurrent integration executed on GPU is between one and two orders of magnitude faster than sequential integration performed on CPU, for two dimensional problems up to one million degrees of freedom.

Acknowledgments The work reported in this paper was supported by the Polish National Science Center grant no. DEC-2012/07/B/ST6/01229.

Bibliography

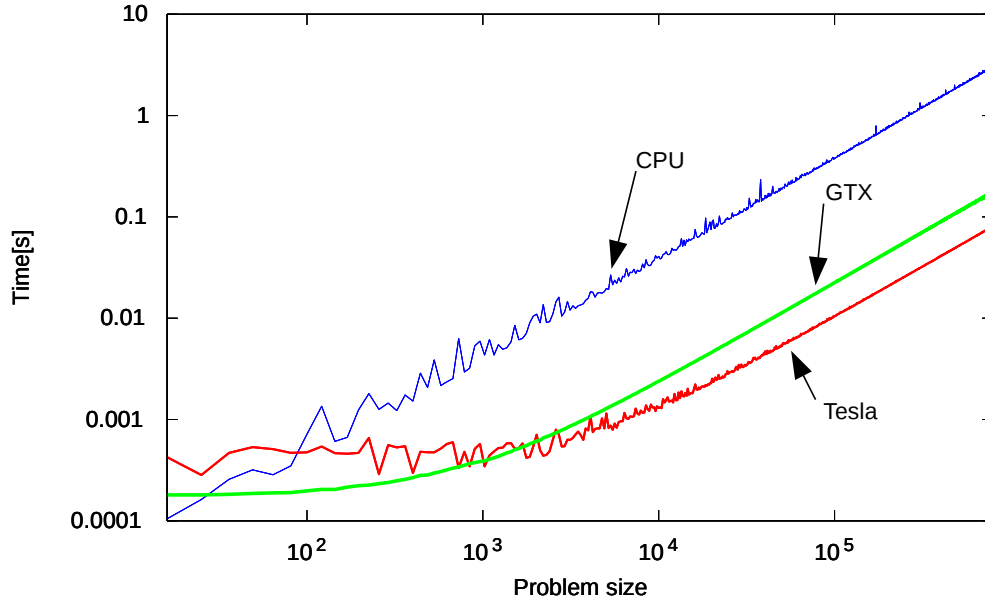


Figure 8: Comparison of GPU and CPU integration time for quadratic B-splines

- [1] J. A. Cottrell, T. J. R. Hughes, J. Bazilevs, Isogeometric Analysis. Toward Integration of CAD and FEA, Wiley, (2009).
- [2] Y. Bazilevs, L. Beirao da Veiga, J.A. Cottrell, T.J.R. Hughes and G. Sangalli, Isogeometric analysis: Approximation, stability and error estimates for h-refined meshes, Mathematical Methods and Models in Applied Sciences,16 (2006) 1031-1090.
- [3] L. Demkowicz, Computing with hp Adaptive Finite Elements. Volume 1. Two Dimensional Elliptic and Maxwell Problems, Chapman & Hall / CRC Press, (2006).
- [4] L. Demkowicz, J. Kurtz, D. Pardo, M. Paszynski, W. Rachowicz, A. Zdunek, Computing with hp Adaptive Finite Elements. Volume 2. Frontiers: Three Dimensional Elliptic and Maxwell Problems with Applications, Chapman & Hall / CRC Press, (2007).
- [5] N. Collier, D. Pardo, L. Dalcin, M. Paszynski, V. Calo, The cost of continuity: A study of the performance of isogeometric finite elements

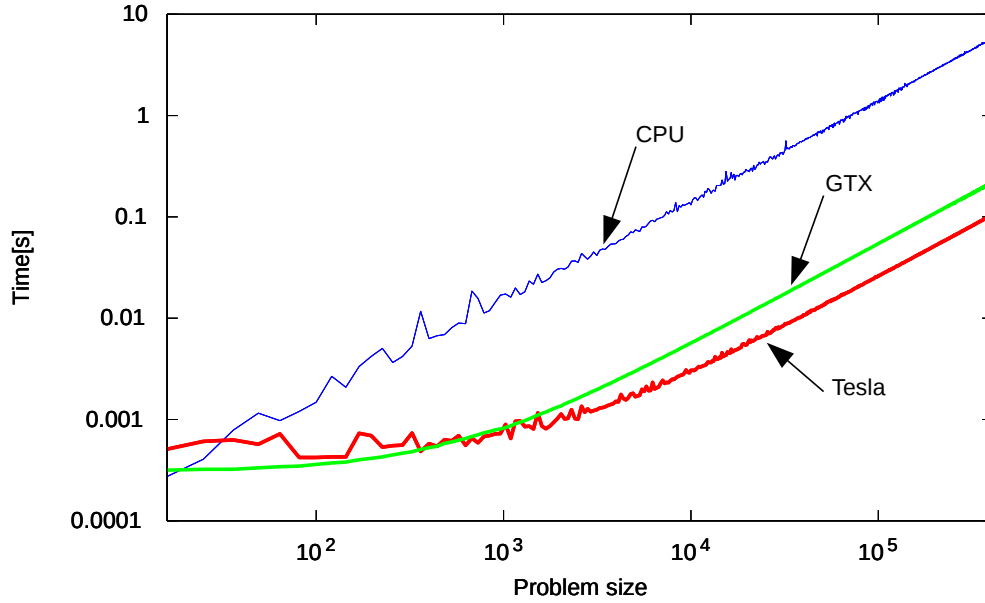


Figure 9: Comparison of GPU and CPU integration time for cubic B-splines

using direct solvers, *Computer Methods in Applied Mechanics and Engineering*, 213-216 (2012) 353-361.

- [6] N. Collier, L. Dalcin, D. Pardo, V. Calo, The cost of continuity: performance of iterative solvers on isogeometric finite elements, *SIAM Journal for Scientific Computing*, 35(2) (2013) A767-A784.
- [7] M. Wozniak, K. Kuznik, M. Paszynski, V. Calo, D. Pardo, Computational cost estimates for parallel shared memory isogeometric multifrontal solvers, *Computers and Mathematics with Applications*, 67(10) (2014) 1864-1883.
- [8] V. Diekert, G. Rosenberg, *The book of traces*, Word Scientific (1995).
- [9] F. Kruzel, K. Banas, Vectorized OpenCL implementation of numerical integration for higher order finite elements, *Computers & Mathematics with Applications* 66 (10) (2013) 2030-2044.
- [10] K. Banas, P. Plaszewski, P. Maciol, Numerical integration on GPUs for higher order finite elements, *Computers & Mathematics with Applications* 67 (6) (2014) 1319-1344.

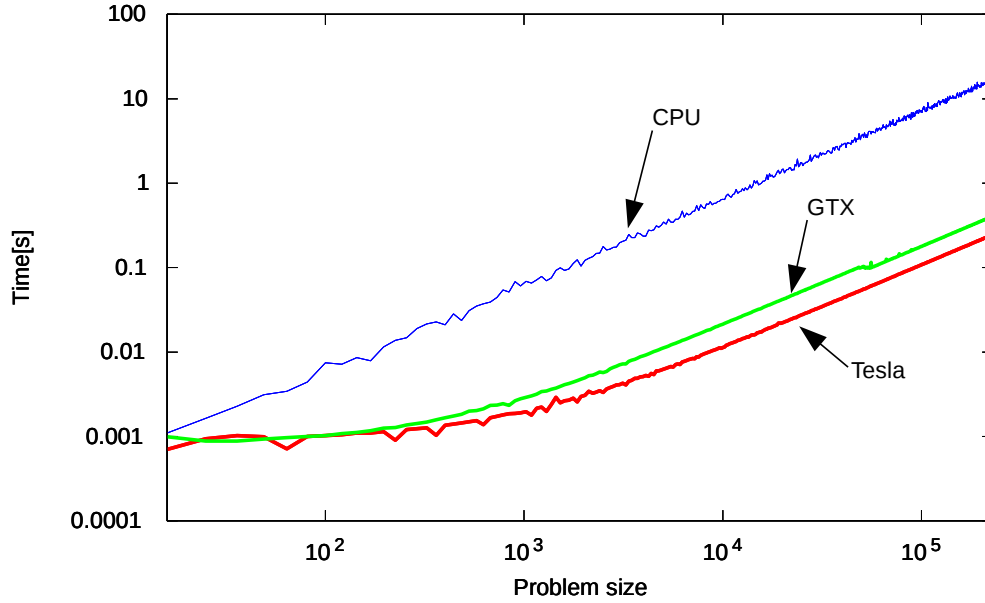


Figure 10: Comparison of GPU and CPU integration time for quartic B-splines

- [11] I. S. Duff and J. K. Reid, *The Multifrontal Solution of Indefinite Sparse Symmetric Linear* ACM Transaction on Mathematical Software, 9(3) (1983) 302-325.
- [12] I. S. Duff and J. K. Reid, *The multifrontal solution of unsymmetric sets of linear systems* SIAM Journal on Scientific Computing, 5 (1984) 633-641.
- [13] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, Multifrontal parallel distributed symmetric and unsymmetric solvers, Computer Methods in Applied Mechanics and Engineering 184 (2000) pp. 501-520.
- [14] P. R. Amestoy, I. S. Duff, J. Koster, J.-Y. L'Excellent, A fully asynchronous multifrontal solver using distributed dynamic scheduling, SIAM Journal of Matrix Analysis and Applications 23 (2001) pp. 15-41.
- [15] P. R. Amestoy, A. Guermouche, J.-Y. L'Excellent, J.-Y. Pralet, Hybrid

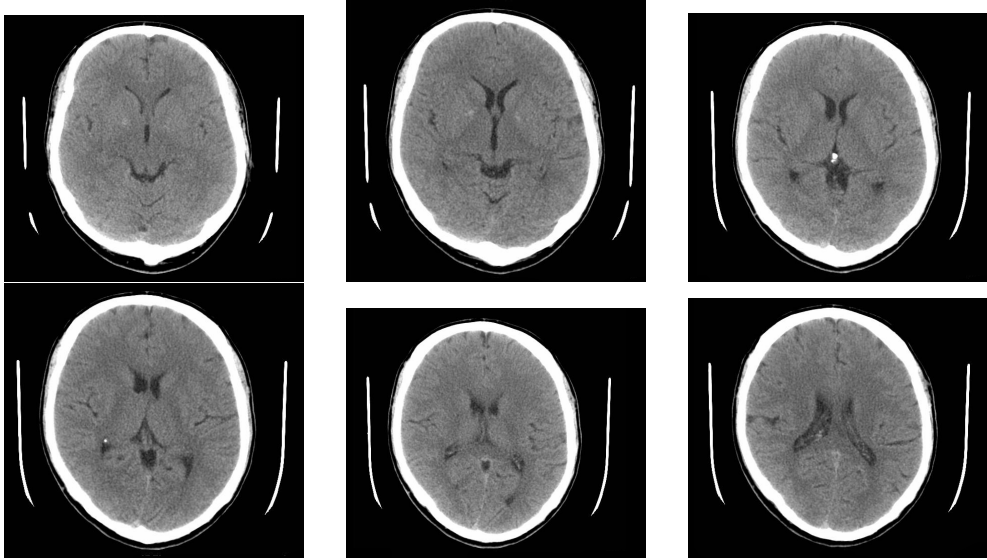


Figure 11: MRI scan data

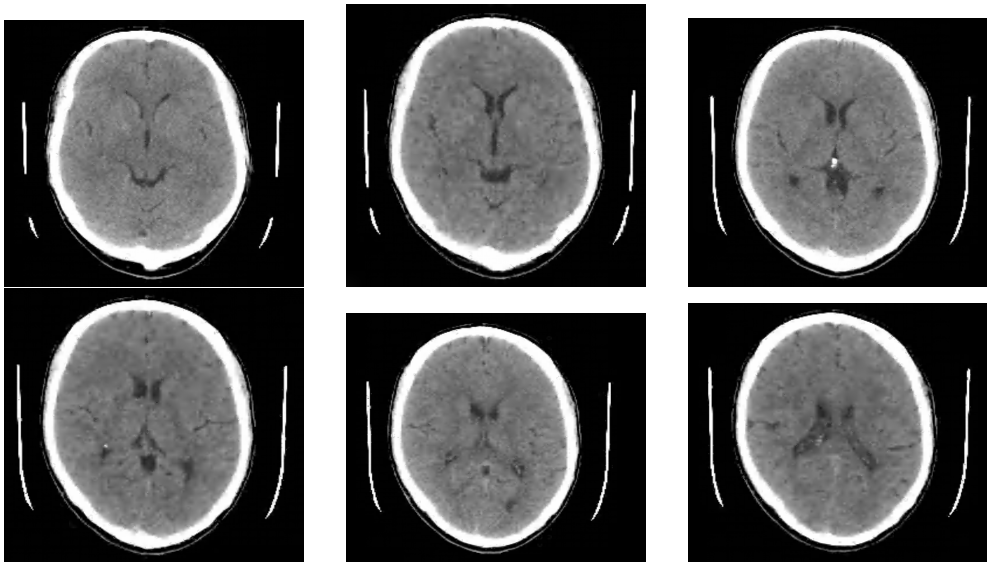


Figure 12: Isogeometric L^2 -projections

- scheduling for the parallel solution of linear systems, *Parallel Computing* 32, 2 (2006) pp. 136-156.
- [16] MUMPS: <http://www.enseeiht.fr/lima/apo/MUMPS> version 4.7.3
 - [17] L. Gao, V. Calo, Fast isogeometric solvers for explicit dynamics, *Computer Methods in Applied Mechanics and Engineering*, 274 (2014) 19-41.
 - [18] S. Beuchler, V. Pillwein, S. Zaglmayr, Fast Summation Techniques for Sparse Shape Functions in Tetrahedral *hp*-FEM, *Domain Decomposition Methods in Science and Engineering, Lecture Notes in Computational Science and Engineering*, 91 (2013) 511-518.
 - [19] T. J. R. Hughes, A. Reali, G. Sangalli, Efficient quadrature for NURBS-based isogeometric analysis, *Computer methods in applied mechanics and engineering*, 199(5) (2010) 301-313.
 - [20] X. S. Li, An Overview of SuperLU: Algorithms, Implementation and User Interface, *TOMS Transactions on Mathematical Software*, 31(3) (2005) 302-325.
 - [21] X.S. Li, J.W. Demmel, J.R. Gilbert, iL. Grigori, M. Shao, I. Yamazaki, *SuperLU Users' Guide*, Lawrence Berkeley National Laboratory, LBNL-44289 <http://crd.lbl.gov/xiaoye/SuperLU/> (1999).
 - [22] P. Hnon, P. Ramet, J. Roman, *PaStiX: A High-Performance Parallel Direct Solver for Sparse Symmetric Definite Systems*, *Parallel Computing*, 28(2) (2002) 301-321.
 - [23] S. Balay, S. Abhyankar, M. F. Adams, J. Brown, P. Brune, K. Buschelman, V. Eijkhout, W. D. Gropp, D. Kaushik, M. G. Knepley, L. Curfman McInnes, K. Rupp, B. F. Smith, H. Zhang, *PETSc Web Page*, <http://www.mcs.anl.gov/petsc> (2014)
 - [24] S. Balay, S. Abhyankar, M. F. Adams, J. Brown, P. Brune, K. Buschelman, V. Eijkhout, W. D. Gropp, D. Kaushik, M. G. Knepley, L. Curfman McInnes, K. Rupp, B. F. Smith, H. Zhang, *PETSc User Manual*, Argonne National Laboratory ANL-95/11 - Revision 3.4 (2013)

- [25] S. Balay, W. D. Gropp, L. Curfman McInnes, B. F. Smith, *Efficient Management of Parallelism in Object Oriented Numerical Software Libraries*, Modern Software Tools in Scientific Computing, Editors E. Arge, A. M. Bruaset and H. P. Langtangen (1997) Birkhäuser Press.
- [26] D. Goik, K. Jopek, M. Paszyski, A. Lenharth, D. Nguyen, K. Pingali, Graph Grammar based Multi-thread Multi-frontal Direct Solver with Galois Scheduler. *Procedia Computer Science*, 29 (2014) 960-969.
- [27] H. AbouEisha, M. Moshkov, V. Calo, M. Paszynski, D. Goik, K. Jopek, Dynamic Programming Algorithm for Generation of Optimal Elimination Trees for Multi-frontal Direct Solver Over H-refined Grids, *Procedia Computer Science*, 29 (2014) 947-959.
- [28] P. Gurgul, A Linear Complexity Direct Solver for H-Adaptive Grids with Point Singularities, *Procedia Computer Science*, 29 (2014) 1090-1099.
- [29] P. G. Schmitz and L. Ying. A fast direct solver for elliptic problems on general meshes in 2D. *Journal of Computational Physics*, 231:1314-1338, (2012).
- [30] P. G. Schmitz and L. Ying. A fast multifrontal solver for 3D elliptic problems using hierarchical matrices. *Journal of Computational Physics*, (2013), submitted.